

Polimorfismo em Go

O Problema Universal

Múltiplos Tipos, Mesma Operação

```
// Queremos fazer isso:  
rectangle.Area() // → 72.0  
circle.Area()    // → 314.15...  
triangle.Area()  // → 36.0  
  
// Como uma função pode aceitar todos esses tipos?
```

Três linguagens, três abordagens:

- Java: Herança + declaração explícita
- Python: Duck typing sem garantias
- Go: Satisfação implícita + type safe

Java — Herança e **implements**

```
// 1. Definir interface
public interface Shape {
    double area();
}

// 2. Declarar EXPLICITAMENTE que implementa
public class Rectangle implements Shape { // ← obrigatório!
    private double width, height;

    @Override // ← precisa declarar override
    public double area() {
        return width * height;
    }
}

// 3. Usar
Shape shape = new Rectangle(); // ✅ funciona
shape.area();
```

Python — Duck Typing

```
# 1. Sem definição de interface formal!
class Rectangle:
    def area(self):
        return self.width * self.height

class Circle:
    def area(self):
        return math.pi * self.radius ** 2

# 2. Usar – aceita QUALQUER coisa com area()
def check_area(shape, expected):
    assert shape.area() == expected

check_area(Rectangle(), 72.0) # ✅ funciona
check_area("string", 10)     # ❌ erro em RUNTIME!
```

Características: Flexível, sem garantias, erro só em runtime

Go — Satisfação Implícita

```
// 1. Definir interface
type Shape interface {
    Area() float64
}

// 2. Implementar – SEM declarar!
type Rectangle struct { Width, Height float64 }

func (r Rectangle) Area() float64 { // ← não diz "implements"
    return r.Width * r.Height
}

// 3. Usar
var shape Shape = Rectangle{Width: 12, Height: 6} // ✅ funciona
shape.Area()

var shape Shape = "string" // ❌ erro em COMPILE TIME!
```

Características: Implícito, type-safe, verificado pelo compilador

Comparação

Aspecto	Java	Python	Go
Declaração	<code>implements Shape</code>	Nenhuma	Nenhuma 
Type Safety	 Compile time	 Runtime	 Compile time
Flexibilidade	 Rígido	 Total	 Alta
Acoplamento	 Alto	 Baixo	 Baixo
Erro detectado	Compilação	Execução	Compilação

Go combina o melhor dos dois mundos:

- Duck typing (flexível) + Type safety (seguro)

O Que Go NÃO Tem

Herança de Classes

```
type Square extends Rectangle { } // ✗ não existe
```

Overriding (Sobreescreita)

```
// Não há conceito de @Override  
// Cada tipo tem seus próprios methods
```

Overloading (Sobrecarga)

```
func Hello(name string) string { }  
func Hello(name string, hour int) string { } // erro!
```

Lembra do Olá, Mundo?

Por Que Functional Options?

Problema sem overloading:

```
// ❌ Go não permite:  
func Hello(name string) string  
func Hello(name string, hour int) string  
func Hello(name string, hour int, region string) string  
// 3 parâmetros = 8 versões possíveis! 🙄
```

Solução — Functional Options:

```
// ✅ Uma função, infinitas combinações:  
func Hello(options ...HelloOption) string  
  
Hello()  
Hello(WithName("João"))  
Hello(WithName("João"), WithHour(20), WithRegion(RegionRN))
```


Overloading vs Functional Options

Java com Overloading:

```
public String hello(String name) { ... }  
public String hello(String name, int hour) { ... }  
public String hello(String name, String region) { ... }  
public String hello(int hour, String region) { ... }  
public String hello(String name, int hour, String region) { ... }  
// Adicionar 1 param → DOBRA o número de funções!
```

Go com Functional Options:

```
func Hello(options ...HelloOption) string { ... }  
// Adicionar param → criar WithFoo(), pronto!
```

Resultado: Código mais limpo, flexível e extensível

Para Web — http.Handler

```
// Isso é uma interface!
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}

// Seu handler:
type MyHandler struct {
    db *DB
}

func (h *MyHandler) ServeHTTP(w ResponseWriter, r *Request) {
    // processar HTTP request
}

// MyHandler satisfaz Handler
```